

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
Asignatura:	Sistemas Distribuidos				
Título de la práctica:	Seguridad en Sistemas Distribuidos (LogiMarket Perú)				
Número de la práctica:	11	Año lectivo:	2026	Nro. semestre:	A
Fecha de presentación:	2026-06-28	Hora de presentación:	23:59		
Integrante(s): - Barrios Medina, Mathias Alonso - Huacani Jara Denise Andrea - Hanco Mullisaca, Sergio Danilo - Pacheco Palo, Fabiana Francinet - Quispe Condori, Alvaro Raul				NOTA:	Nota colocada por el docente
Docente: Docente de Sistemas Distribuidos					

SOLUCIÓN Y RESULTADOS							
1. Solución de Actividades y Resultados							
Repositorio de GitHub: https://github.com/uns-a-semester-2026-A/sis_dis_lab/tree/main/lab11							
1.1. Actividad 1 – Identificación y Análisis Cuantitativo de Amenazas							
En esta actividad se identificaron los activos de LogiMarket Perú, se analizaron los incidentes ocurridos y se diseñó una matriz de riesgos utilizando una metodología cuantitativa de evaluación de riesgos. Para ello, se aplicó la fórmula estándar recomendada en la guía metodológica del NIST SP 800-30 [1]:							
$R = \text{Probabilidad (P)} \times \text{Impacto (I)}$							
Tanto la probabilidad de ocurrencia como el impacto en el negocio se valoran en una escala del 1 al 5. El riesgo resultante se clasifica en cuatro niveles críticos: Bajo (1–4), Medio (5–12), Alto (13–20) y Crítico (21–25). Esta escala ayuda a registrar y priorizar las acciones correctivas inmediatas según la severidad del activo comprometido.							
1.1.1. Matriz de Riesgos							
ID Ac-tivo	Activo Crítico	Amenaza Identifi-cada	P	I	R	Nivel	Mecanismo de Control Propuesto
ACT-01	Portal Web	Acceso no autorizado mediante el robo o compartición de credencia-	4	4	16	ALTO	Autenticación multifactor (MFA/TOTP) y control centralizado de sesiones activas.

ID Ac-tivo	Activo Crítico	Amenaza Identifi-cada	P	I	R	Nivel	Mecanismo de Control Propuesto
		les de administra-ción.					
ACT-02	App Móvil	Interceptación de datos en tránsito y suplantación de la API.	3	4	12	MEDIO	Implementación de SSL/TLS Pinning en el cliente móvil.
ACT-03	API Gateway / Mi-croservicios	Bypass del Gate-way y consumo di-recto no autoriza-do de microservi-cios internos.	5	5	25	CRÍTI-CO	mTLS interno y va-lidación autónoma de la firma de JWT en cada microservi-cio.
ACT-04	DB Inventario	Modificación ma-nual maliciosa de datos sin dejar ras-tro de autoría en el sistema.	4	5	20	ALTO	Habilitación de au-ditoría a nivel de base de datos y logs inmutables JSON.
ACT-05	Gateway Pagos	Interceptación de credenciales y da-tos financieros por canales inseguros.	3	5	15	ALTO	Configuración obli-gatoria de TLS 1.3, deshabilitando co-nexiones HTTP pla-nas.
ACT-06	MS Logística	Caída del servicio de despachos de-bido a sobrecarga de peticiones mali-ciosas (DoS).	4	4	16	ALTO	Políticas de Rate Li-miting por dirección IP y Token de clien-te en el Gateway.

El análisis de riesgos revela que el bypass de los microservicios internos representa el riesgo más crítico (**R = 25**), ya que anula cualquier medida perimetral intermedia y expone la lógica interna del negocio.

1.2. Actividad 2 – Diseño de Autenticación Segura (MFA + Contraseñas)

Para resolver de manera exhaustiva el **Incidente 5** (credenciales de administrador compartidas e inicios de sesión desde ubicaciones geográficamente incoherentes), se propone un diseño de arquitectura de autenticación en dos fases y una política activa de gestión de sesiones.

1.2.1. Flujo Arquitectónico de Autenticación

El establecimiento de sesión segura se modela a través de la secuencia presentada en la Figura 1.

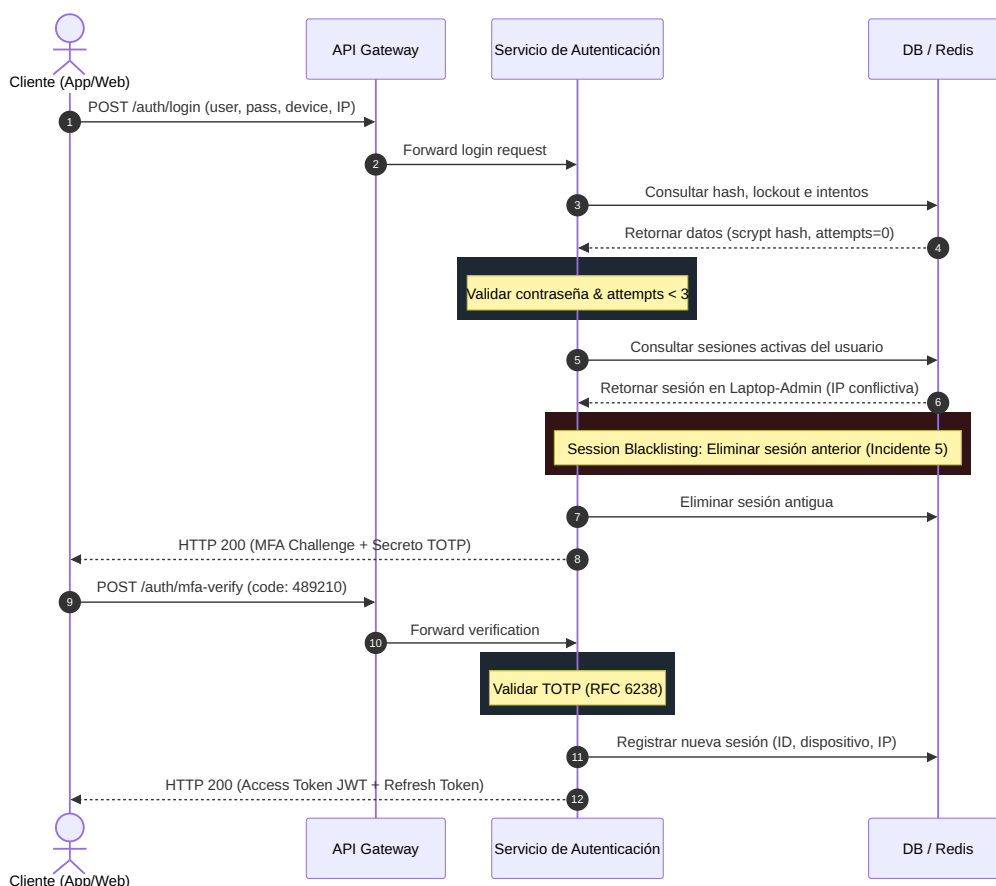


Figura 1: Diagrama de secuencia del flujo de autenticación y revocación concurrente.

1.2.2. Descripción Detallada del Diseño y Políticas Propuestas

- Almacenamiento de Credenciales (Hashing Iterativo con Scrypt):** Las contraseñas de los usuarios no se guardan directamente. En su lugar, se procesan mediante la función de derivación de claves scrypt. A diferencia de algoritmos de hashing rápidos como SHA-256, scrypt requiere una cantidad configurable de memoria RAM y tiempo de CPU para calcular cada hash. Esto hace inviable computacionalmente que un atacante intente descifrar los hashes utilizando hardware paralelo dedicado (como GPUs o ASICs) en caso de sufrir una brecha en la base de datos de usuarios.
- Desafío de Segundo Factor (MFA - TOTP):** Se implementa el estándar industrial **RFC 6238** (Time-Based One-Time Password). En el registro de usuario, el servidor genera una clave semilla Base32 única compartida, que el administrador escanea mediante una aplicación autenticadora (ej. Google Authenticator). Cada 30 segundos, el dispositivo genera un código único de 6 dígitos basado en el tiempo Epoch actual y la semilla. El servidor recalcula el valor usando el mismo algoritmo y tiempo, aceptando una ventana de tolerancia de ± 1 intervalo (90 segundos en total) para evitar fallas por desincronización de relojes.
- Política de Bloqueo de Cuenta (Lockout):** Para disuadir los ataques de fuerza bruta en la fase de contraseña, se establece que tras **3 intentos fallidos consecutivos**, la cuenta se bloquea temporalmente durante

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

un periodo incremental (comenzando por 30 segundos). El servidor rechaza inmediatamente cualquier intento posterior en este lapso, mitigando el uso excesivo de recursos de CPU provocado por el descifrado script.

4. **Revocación Activa de Sesiones Concurrentes (Blacklisting):** Al iniciar una sesión válida, el proveedor de identidad registra en un almacén en memoria (ej. clúster de Redis) el ID de sesión, el usuario, la huella digital del dispositivo y la dirección IP geográfica. Si el usuario intenta loguearse desde otro dispositivo o desde un rango de IP sospechoso, el sistema detecta la anomalía de concurrencia. De manera inmediata, el servidor aplica un mecanismo de **Session Blacklisting**, invalidando el token anterior y forzando a todas las conexiones previas a re-autenticarse con MFA. Esto impide que credenciales compartidas o robadas se utilicen simultáneamente en diferentes ubicaciones físicas.

1.3. Actividad 3 – Seguridad de Comunicaciones (OpenSSL + TLS 1.3)

Para solucionar el **Incidente 2** (interceptación pasiva de credenciales y datos de pagos enviados en texto plano HTTP), se estableció una Infraestructura de Llave Pública (PKI) local que cifra el canal de comunicación extremo a extremo.

1.3.1. Creación de Certificados locales con OpenSSL

Se desarrolló el script `generate_certs.sh` para crear una Entidad Certificadora (CA) raíz corporativa autofirmada y emitir el certificado del servidor para `logimarket.local`. Para cumplir con las directrices de seguridad de las librerías modernas de comunicación, el certificado de servidor se generó incluyendo extensiones de **Subject Alternative Name (SAN)**, mapeando el dominio local y la IP de bucle de retorno:

```
# Configuración de extensiones SAN
subjectAltName = @alt_names
[alt_names]
DNS.1 = logimarket.local
IP.1 = 127.0.0.1
```

1.3.2. Verificación de Conexión TLS 1.3

El servidor seguro y el cliente escritos en Python se configuraron para forzar como protocolo criptográfico mínimo **TLS 1.3** de la IETF [2] y suites de cifrado simétrico avanzadas. Los resultados de la validación se capturaron con la herramienta de la terminal en la Figura 2:

- **Intento 1 (Sin CA de confianza):** El cliente interrumpe el handshake y arroja un error controlado [SSL: CERTIFICATE_VERIFY_FAILED], demostrando que el cliente rechaza de forma autónoma certificados desconocidos, protegiéndose contra ataques de interceptación o suplantación de identidad (MitM).
- **Intento 2 (Con CA de confianza cargada):** Se establece exitosamente el canal cifrado negociando la suite TLS_AES_256_GCM_SHA384 con 256 bits de seguridad criptográfica.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

```

> python3 actividad3_comunicaciones/run_ssl_test.py
127.0.0.1 - - [28/Jun/2026 22:25:31] "GET / HTTP/1.1" 200 -
=====
      INICIO DE PRUEBAS AUTOMATIZADAS DE SEGURIDAD - ACTIVIDAD 3
=====

[Test] Iniciando servidor HTTPS seguro en segundo plano...

[+] Servidor HTTPS seguro corriendo en https://localhost:8443
[+] Forzando protocolo mínimo: TLSv1.3
[+] Certificado cargado: /home/alvaro9rqc/1_Pacha/1-unsalab/1-unsalab/actividad3_comunicaciones/certs/server.crt
[*] Presione Ctrl+C para detener el servidor...

[Test] Iniciando pruebas de conexión con secure_client...
-----
[!] Conectando SIN Validación de CA Raíz (Esperando error de confianza)
-----
[X] Error de conexión: <urlopen error [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: unable to get local issuer
-----

[+] Conectando CON Validación de CA Raíz: /home/alvaro9rqc/1_Pacha/1-unsalab/1-unsalab/actividad3_comunicaciones
-----
[✓] CONEXION EXITOSA de extremo a extremo!
- Servidor: https://localhost:8443
- Version TLS negociada: TLSv1.3
- Suite de Cifrado: TLS_AES_256_GCM_SHA384
- Payload Recibido:
{
  "status": "CONEXION_SEGURA_OK",
  "mensaje": "Bienvenido al canal seguro de comunicaciones de LogiMarket Peru",
  "detalles_tls": {
    "version_protocolo": "TLSv1.3",
    "cipher_suite": "TLS_AES_256_GCM_SHA384",
    "bits_seguridad": 256
  },
  "activo_protegido": "Gateway de Pagos & API de Transacciones"
}

=====
      FIN DE PRUEBAS - APAGANDO SERVIDOR
=====

```

Figura 2: Captura de la ejecución del handshake y validación de la cadena de confianza TLS 1.3.

1.4. Actividad 4 – Protección de APIs (JWT + OAuth 2.0 + API Gateway)

Para solucionar el bypass de seguridad de microservicios (**Incidente 1**) y las caídas por inundación DoS (**Incidente 3**), se estructuró una arquitectura de protección perimetral e interna fundamentada en OAuth 2.0 y firmas criptográficas distribuidas.

1.4.1. Arquitectura y Flujo Técnico de APIs

La interacción coordinada entre el Cliente, el API Gateway, el Servidor de Autorización y el Microservicio backend se esquematiza en la Figura 3.

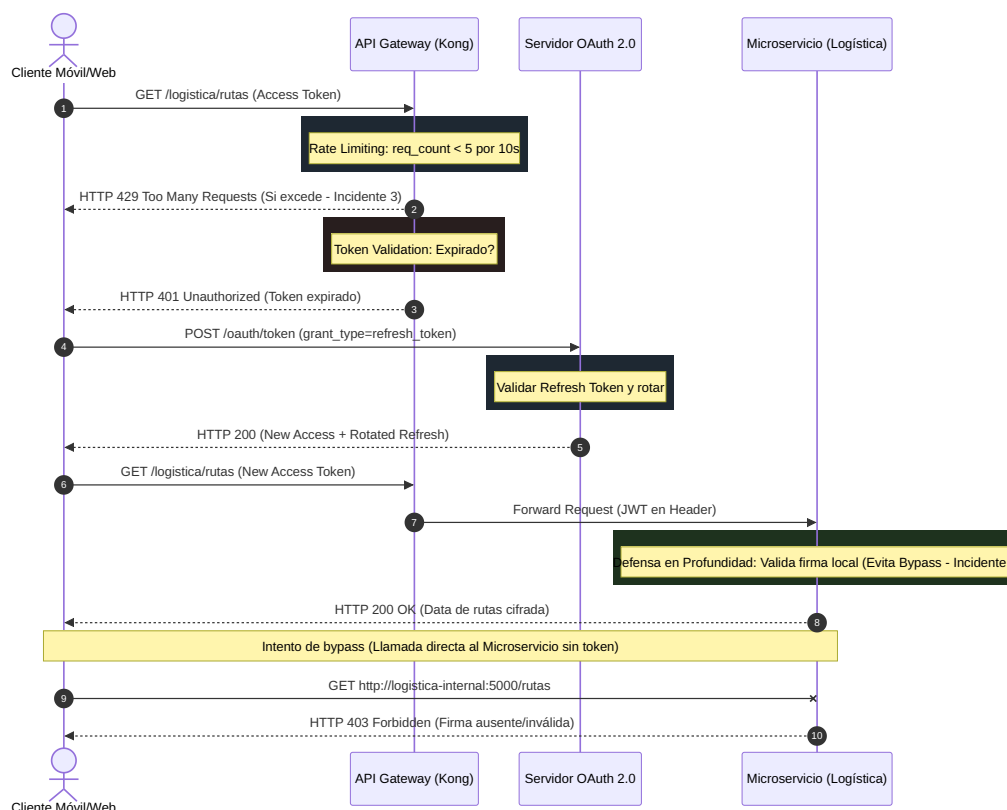


Figura 3: Diagrama de secuencia de la arquitectura de API Gateway y validación distribuida.

1.4.2. Análisis de los Componentes Propuestos

- API Gateway como Punto de Entrada Único:** El API Gateway (como Kong o APISIX) actúa como el proxy inverso expuesto al exterior. Se encarga de centralizar las peticiones e interceptar ataques. Implementa un middleware de **Rate Limiting** que rastrea la tasa de peticiones por dirección IP y token del cliente. Al superar la cuota máxima permitida (ej. 5 peticiones cada 10 segundos), el Gateway corta la petición devolviendo un código de error **HTTP 429 Too Many Requests**, protegiendo la disponibilidad operativa de los servicios backend ante inundaciones accidentales o intencionadas de red DoS (Incidente 3).
- Estructura y Autorización basada en JSON Web Tokens (JWT):** La autenticación se realiza de manera centralizada en el servidor OAuth 2.0. Al loguearse con éxito, se emite un **Access Token** en formato JWT auto-contenido que expira rápidamente (ej. 15 minutos). El payload contiene claims específicos como emisor (iss), usuario (sub), rol (role), fecha de emisión (iat) y fecha de expiración (exp). El token se firma usando algoritmos asimétricos robustos (como RSA256 o ES256) empleando la llave privada del Servidor de Autenticación.
- Validación Descentralizada de Firmas (Defensa en Profundidad):** Para evitar el bypass del API Gateway (Incidente 1), se establece que los microservicios internos **no confían implícitamente en las peticiones que les llegan**. De forma autónoma, cada microservicio valida localmente la firma criptográfica del JWT adjunto en la cabecera Authorization. Para esto, obtienen la llave pública del Servidor de Autenticación

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

(a través de endpoints seguros JWKS) y validan que el token no haya sido modificado ni esté expirado. Si un atacante intenta saltarse el Gateway y llamar directamente al puerto del microservicio interno, este detecta la ausencia de firma válida y bloquea la llamada respondiendo con un error **HTTP 403 Forbidden**.

4. **Ciclo de Vida Seguro y Refresh Tokens:** Para que el usuario no deba reintroducir sus credenciales constantemente tras la expiración corta del Access Token, se emite un **Refresh Token** de mayor duración. Al ser utilizado para renovar el Access Token en el Auth Server, el sistema aplica la política de **rotación de refresh tokens**: el token de refresco viejo se anula en la base de datos y se emite uno totalmente nuevo junto al nuevo Access Token. Si el token viejo fuese interceptado o reutilizado de manera maliciosa, el servidor de autorización detecta el intento de reuso, invalida inmediatamente toda la familia de tokens asociada a ese usuario y bloquea la sesión por seguridad.

1.5. Actividad 5 – Sistema de Auditoría y Logs Centralizados

Para mitigar el **Incidente 4** (ausencia de pistas de auditoría tras modificaciones manuales de stock), se diseñó un pipeline de auditoría centralizado que recolecta, indexa y alerta sobre eventos de seguridad en tiempo real.

1.5.1. Estructura del Pipeline de Auditoría

El flujo de eventos desde los microservicios hasta su destino de retención e indexación se modela en la Figura 4.

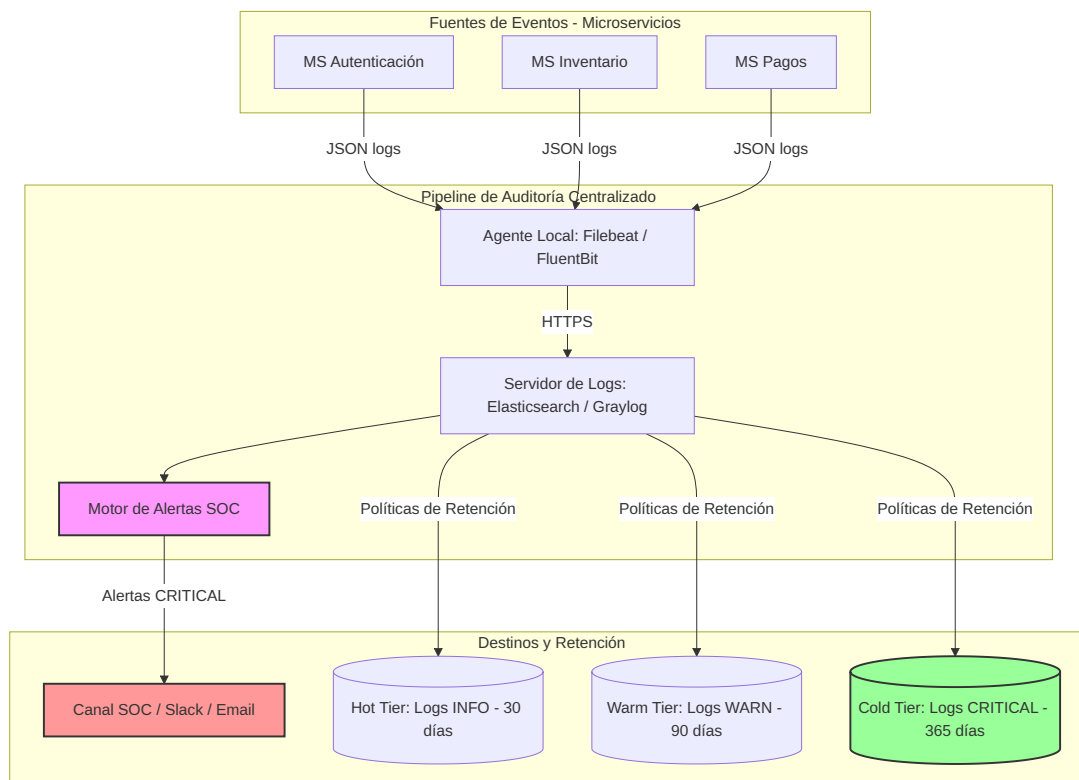


Figura 4: Diagrama de flujo del pipeline de auditoría y niveles de retención.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

1.5.2. Descripción del Pipeline e Indexación

- 1. Generación de Eventos:** Cada microservicio genera registros de eventos en un formato uniforme JSON estructurado. Se implementa una taxonomía de 9 campos obligatorios: @timestamp (ISO 8601), service_name, severity (INFO, WARNING, CRITICAL), event_id, actor (usuario/proceso responsable), action, status (SUCCESS/FAIL), ip_address y details. Cualquier operación crítica (como modificar stock) se cataloga obligatoriamente con severidad CRITICAL.
- 2. Agente de Envío Local:** Agentes ligeros como Filebeat o FluentBit leen localmente los logs en JSON Lines y los transmiten por canales HTTPS seguros hacia el agregador.
- 3. Agregador Centralizado (SIEM / Elasticsearch):** Indexa la información facilitando búsquedas inmediatas y análisis forense.
- 4. Alertas en Tiempo Real:** Un motor de reglas de seguridad evalúa constantemente los logs indexados. Al detectarse un evento clasificado como CRITICAL (por ejemplo, una alteración manual o un bypass de API fallido), el sistema dispara una alerta inmediata a los canales de monitoreo del SOC (ej. Slack, PagerDuty, Email) para la intervención de los administradores.
- 5. Políticas de Retención de Logs:**
 - *Hot Tier (INFO):* Logs almacenados en disco de alto rendimiento por 30 días para consultas operativas rápidas; depurados automáticamente al expirar.
 - *Warm Tier (WARNING):* Logs almacenados en almacenamiento comprimido de costo medio por 90 días para auditoría interna regular.
 - *Cold Tier (CRITICAL):* Logs de auditoría de seguridad archivados en frío e inmutables durante 365 días para cumplimiento regulatorio y análisis post-incidentes forenses.

2. Cuestionario de Análisis Crítico

- 1. Si implementamos un sistema MFA, pero aún detectamos accesos no autorizados a cuentas críticas de la empresa, ¿qué factores (humanos, tecnológicos u organizacionales) podrían estar fallando y qué medidas propondrías?

A pesar de que el MFA reduce sustancialmente el riesgo de compromiso de credenciales, los atacantes han desarrollado técnicas sofisticadas para eludirlo. Las fallas analizadas se dividen en tres áreas:

A. Factores Humanos:

- **Fatiga de MFA (Prompt Bombing):** Envío masivo de solicitudes de aprobación push al teléfono del usuario en horarios inusuales, buscando que apruebe la sesión por cansancio o distracción.
- **Phishing Adversario (AitM - Adversary-in-the-Middle):** Uso de herramientas proxy (como Evilginx) que interceptan la clave, el token OTP y la cookie de sesión del navegador en tiempo real, puenteando el MFA.

B. Factores Tecnológicos:

- **Secuestro de Sesiones (Session Hijacking):** Ataques que sustraen las cookies de sesión del navegador del cliente (malware **infostealer**). Al no estar la cookie enlazada a la identidad del dispositivo o IP de origen, el atacante la utiliza en su propio navegador, saltándose el login.
- **SMS vulnerable:** Uso de canales inseguros propensos a ataques de **SIM Swapping** o interceptación de red SS7.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

- **Ausencia de Contexto Geográfico/Temporal:** Aceptar accesos válidos sin verificar la imposibilidad física de traslado (sesiones concurrentes a larga distancia).

C. Factores Organizacionales:

- **Falta de Políticas BYOD:** Permitir accesos a consolas de producción desde computadoras personales sin controles mínimos de seguridad.

Mitigaciones Recomendadas:

1. **MFA Resistente a Phishing:** Adoptar llaves criptográficas físicas FIDO2/WebAuthn o llaves passkeys integradas.
 2. **Number Matching:** Obligar al usuario a ingresar en la app autenticadora un número de 2 dígitos mostrado en la pantalla de login de la PC.
 3. **Autenticación Adaptativa (Basada en Riesgo):** Evaluar geolocalización e IP para bloquear la sesión concurrente sospechosa de inmediato, tal como se implementó en la **Actividad 2**.
 4. **Session Binding y Cookies Efímeras:** Enlazar la sesión a la huella digital del dispositivo y reducir el tiempo de vida de los tokens.
- 2. Evalúa la siguiente afirmación: «El cifrado de todas las comunicaciones en tránsito (HTTPS/TLS) y de los datos almacenados (at-rest) genera un impacto inaceptable en el rendimiento de un sistema distribuido de gran escala». ¿Es verdadera o falsa? Argumenta con datos técnicos.

La afirmación es **falsa** en el contexto tecnológico moderno. Si bien el cifrado introduce un costo computacional y latencia adicional en sistemas antiguos, la evolución de los protocolos y el hardware actual hace que este overhead sea prácticamente insignificante:

A. Cifrado en Tránsito (HTTPS/TLS):

- **Aceleración por Hardware (AES-NI):** Los procesadores modernos implementan el conjunto de instrucciones AES-NI a nivel físico, permitiendo que el cifrado simétrico se ejecute de manera casi instantánea y directa en el silicio, reduciendo el overhead de CPU a niveles menores al 1% en cargas normales.
- **Optimización de TLS 1.3:** Como se detalla en la especificación del protocolo de la IETF [2], TLS 1.3 optimizó el handshake para establecerse en solo **1 RTT** (frente a los 2 RTT de TLS 1.2). Esto reduce a la mitad la latencia de red inicial y el costo de computación asimétrica.
- **HTTP/3 y QUIC:** Habilitan el multiplexado de cientos de peticiones web sobre una única conexión TLS abierta, diluyendo el costo del handshake y eliminando el bloqueo de cabecera de línea de TCP.

B. Cifrado en Reposo (At-Rest):

- **Transparent Data Encryption (TDE):** El cifrado a nivel de bloques de disco o tablas de base de datos se realiza en segundo plano por el kernel o controladores dedicados. En benchmarks de sistemas distribuidos a gran escala, la degradación de rendimiento en operaciones I/O oscila apenas entre el **2% y 5%**.

Conclusión: El impacto de rendimiento no es «inaceptable» sino **despreciable** si se emplean las herramientas correctas. Hoy en día, la seguridad perimetral de tipo Zero Trust (donde todo canal está cifrado) es un estándar industrial mandatorio e implementable a gran escala.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

- 3. Si fueras el Director de Seguridad de Información (CISO) de LogiMarket Perú y contaras con un presupuesto muy limitado, ¿cuál sería tu orden de prioridad para implementar los controles de seguridad del laboratorio? Justifica detalladamente.

Con recursos restringidos, se debe priorizar bajo el principio de **mayor mitigación de riesgos con el menor costo y complejidad de implementación**:

1. Prioridad 1: API Gateway y Autenticación Centralizada (MFA + JWT):

- *Justificación:* Mitiga directamente el **Incidente 1** (bypass de microservicios) y el **Incidente 5** (sesiones concurrentes y credenciales compartidas) que representan las vulnerabilidades de mayor probabilidad e impacto de negocio (Riesgo 25 en la **Actividad 1**).
- *Costo:* Muy bajo. Se pueden configurar Identity Providers de código abierto como **Keycloak** o **Authelia** y Gateways como **APISIX** o **Kong** sin coste de licenciamiento de software. El esfuerzo es puramente configuración e integración.

2. Prioridad 2: Cifrado en Tránsito Externo (TLS 1.3):

- *Justificación:* Resuelve el **Incidente 2** (fuga de datos financieros en texto plano). Es mandatorio para transacciones de pago.
- *Costo:* Cero en licencias. Los certificados de confianza pública se automatizan sin costo mediante entidades gratuitas como **Let's Encrypt**.

3. Prioridad 3: Auditoría Centralizada de Logs (Taxonomía Básica):

- *Justificación:* Resuelve el **Incidente 4** (ausencia de pistas forenses tras una alteración manual).
- *Costo:* Bajo/Medio. El almacenamiento se puede resolver con herramientas libres como **Open-Search** o el stack **ELK** (Elasticsearch, Logstash, Kibana) comunitario. Se requiere disciplinar el desarrollo de logs en formato JSON estructurado.

4. Prioridad 4: mTLS Interno y Cifrado en Reposo:

- *Justificación:* Protege contra atacantes internos o filtraciones en la red LAN.
- *Costo:* Alto en ingeniería y complejidad. Mantener una malla de servicios (Service Mesh) para automatizar mTLS y encriptar volúmenes de base de datos requiere gran experiencia y recursos humanos. Se posterga a una segunda fase cuando aumente el presupuesto de seguridad.

3. Conclusiones y Lecciones Aprendidas

- Defensa en Profundidad vs. Confianza Perimetral:** Confiar únicamente en que el API Gateway filtre y valide la seguridad es un antipatrón de diseño de sistemas distribuidos. Cada microservicio interno debe verificar criptográficamente la autenticidad y vigencia de las solicitudes de forma autónoma (validación de firma JWT), tal como se demostró en la **Actividad 4**, evitando el bypass del perímetro (Incidente 1).
- Estándares Criptográficos Modernos:** TLS 1.3 e instrucciones de hardware dedicadas (AES-NI) demuestran que el cifrado ya no es una limitación de rendimiento. Su adopción obligatoria es fundamental para mitigar la interceptación de información y credenciales de pago (Incidente 2).
- MFA Resistente y Adaptabilidad:** Los controles de acceso tradicionales basados en contraseñas estáticas y MFA clásico por SMS son vulnerables. El uso de autenticación adaptativa con bloqueo ante fuerza bruta (Lockout) y Session Blacklisting es crítico para neutralizar accesos concurrentes no autorizados desde ubicaciones imposibles (Incidente 5).

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

4. **Logs Estructurados y Auditoría Inmutable:** Para realizar análisis forenses confiables y auditoría tras incidentes de modificación de stock (Incidente 4), los sistemas deben generar logs estructurados (taxonomía JSON) clasificados por niveles de severidad y guardados bajo políticas de retención y rotación automatizadas, garantizando su inmutabilidad frente a accesos internos no autorizados.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] National Institute of Standards and Technology (NIST), «Guide for Conducting Risk Assessments», Special Publication 800-30 Revision 1, 2012.
- [2] E. Rescorla, «The Transport Layer Security (TLS) Protocol Version 1.3», technical report RFC 8446, 2018.